

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

РГУ им. А. Н. Косыгина (Технологии. Дизайн. Искусство.)

Институт информационных технологий и цифровой трансформации

Кафедра «Искусственного интеллекта, прикладной математики и программирования»

КУРСОВАЯ РАБОТА

по дисциплине

«Вероятностное моделирование процессов и систем»

Выполнил: студент группы ИТПМ-124

Молчанов Даниил Максимович

_____ (подпись)

Принял: доцент кафедры ИИПМиП

Романенков Александр Михайлович

_____ (подпись)

Оценка: _____

Дата: _____

Содержание

Введение	2
1 Теоретическая часть	2
1.1 Основные понятия теории вероятностей	2
1.2 Теоремы сложения и умножения вероятностей	2
1.3 Дискретные случайные величины	4
1.4 Функция распределения	5
1.5 Основные дискретные распределения	6
1.5.1 Биномиальное распределение	6
1.5.2 Геометрическое распределение	6
1.5.3 Равномерное дискретное распределение	6
1.5.4 Треугольное дискретное распределение	6
1.6 Комбинаторика	6
1.7 Элементы теории графов	7
1.8 Парадокс дней рождения	7
2 Практическая часть	9
2.1 Выбор технологий и инструментов	9
2.2 Архитектура программного комплекса	9
2.3 Описание реализованных заданий	9
2.3.1 Задание №1. Проверка теоремы сложения вероятностей	9
2.3.2 Задание №2. Моделирование эпидемии	11
2.3.3 Задание №3. Моделирование производственного процесса	13
2.3.4 Задание №4. Двумерное случайное блуждание	17
2.3.5 Задание №5. Одномерное случайное блуждание с возвратом	19
2.3.6 Задание №6. Генерация ступенчатых фигур	21
2.3.7 Задание №7. Парадокс дней рождения и атака на шифр Вернама	24
2.3.8 Задание №9. Условная вероятность для монет	26
2.3.9 Задание №10. Вероятность взрыва бочки	28
2.3.10 Задание №12. Моделирование случайного блуждания	30
Вывод	33
Список использованных источников	33
Приложение	33

Введение

Данная курсовая работа посвящена разработке комплекса приложений для вероятностного моделирования процессов и систем. В работе рассматриваются задачи из различных областей теории вероятностей: комбинаторика, моделирование эпидемий, производственные процессы, случайные блуждания, криптоанализ, дискретные случайные величины и теория графов. Все программные модули реализованы на языке C++ с использованием библиотеки Qt для построения графического интерфейса. В пояснительной записке представлены математические основы, описание архитектуры и алгоритмов, а также результаты тестирования.

1 Теоретическая часть

1.1 Основные понятия теории вероятностей

Определение 1.1.1. Пространством элементарных исходов Ω называется множество всех возможных исходов некоторого случайного эксперимента. Каждый элементарный исход $\omega \in \Omega$ соответствует одному возможному результату эксперимента.

Определение 1.1.2. Случайным событием называется любое подмножество $A \subseteq \Omega$. Говорят, что событие A произошло, если в результате эксперимента наступил элементарный исход $\omega \in A$.

Определение 1.1.3. События называются несовместными, если они не могут произойти одновременно, то есть $A \cap B = \emptyset$.

Определение 1.1.4. Вероятностью называется числовая функция P , определённая на множестве событий $\mathcal{F}$ (σ -алгебре), удовлетворяющая трём аксиомам Колмогорова:

1. Аксиома неотрицательности: $P(A) \geq 0$ для любого события A .
2. Аксиома нормированности: $P(\Omega) = 1$.
3. Аксиома счётной аддитивности: для любой последовательности попарно несовместных событий $A_1, A_2, \dots$ выполняется:

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i).$$

Определение 1.1.5. Если пространство элементарных исходов Ω состоит из N равновероятных исходов, а событию A благоприятствует M исходов, то вероятность события A вычисляется по формуле:

$$P(A) = \frac{M}{N}. \quad (1)$$

Определение 1.1.6. Вероятность события A есть предел частоты его появления в серии из n независимых испытаний при $n \rightarrow \infty$:

$$P(A) = \lim_{n \rightarrow \infty} \frac{m_n}{n}, \quad (2)$$

где m_n — число испытаний, в которых наступило событие A .

1.2 Теоремы сложения и умножения вероятностей

Теорема 1.2.1 (Сложение вероятностей для несовместных событий). Если события A и B несовместны, то вероятность их объединения равна сумме вероятностей:

$$P(A \cup B) = P(A) + P(B). \quad (3)$$

Доказательство. Поскольку A и B несовместны, $A \cap B = \emptyset$. Тогда число исходов, благоприятствующих $A \cup B$, равно сумме исходов, благоприятствующих A и B по отдельности. При классическом определении

вероятности:

$$P(A \cup B) = \frac{M_A + M_B}{N} = \frac{M_A}{N} + \frac{M_B}{N} = P(A) + P(B).$$

В общем случае это следует из аксиомы аддитивности для конечного числа несовместных событий. $\square$

Теорема 1.2.2 (Сложение вероятностей для совместных событий). Для любых двух событий A и B справедлива формула:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B). \quad (4)$$

Доказательство. Представим $A \cup B$ как объединение трёх несовместных событий: $A \setminus B$, $B \setminus A$ и $A \cap B$. Тогда

$$P(A \cup B) = P(A \setminus B) + P(B \setminus A) + P(A \cap B).$$

Выразим $P(A \setminus B) = P(A) - P(A \cap B)$ и $P(B \setminus A) = P(B) - P(A \cap B)$. Подставляя, получаем:

$$P(A \cup B) = P(A) - P(A \cap B) + P(B) - P(A \cap B) + P(A \cap B) = P(A) + P(B) - P(A \cap B).$$

$\square$

Замечание 1.2.1. Для трёх событий формула имеет вид:

$$P(A \cup B \cup C) = P(A) + P(B) + P(C) - P(A \cap B) - P(A \cap C) - P(B \cap C) + P(A \cap B \cap C). \quad (5)$$

Доказательство проводится аналогично с помощью метода включений-исключений.

Определение 1.2.1. Условной вероятностью события A при условии, что событие B произошло ($P(B) > 0$), называется величина:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}. \quad (6)$$

Теорема 1.2.3 (Умножение вероятностей). Для любых событий A и B :

$$P(A \cap B) = P(A) \cdot P(B|A) = P(B) \cdot P(A|B). \quad (7)$$

Доказательство. Следует непосредственно из определения условной вероятности: домножая обе части равенства $P(A|B) = \frac{P(A \cap B)}{P(B)}$ на $P(B)$, получаем $P(A \cap B) = P(B) \cdot P(A|B)$. Аналогично для $P(B|A)$. $\square$

Для произвольного числа событий формула обобщается:

$$P(A_1 \cap A_2 \cap \dots \cap A_n) = P(A_1) \cdot P(A_2|A_1) \cdot P(A_3|A_1 \cap A_2) \cdot \dots \cdot P(A_n|A_1 \cap \dots \cap A_{n-1}). \quad (8)$$

Определение 1.2.2. События A и B называются независимыми, если:

$$P(A \cap B) = P(A) \cdot P(B). \quad (9)$$

События $A_1, A_2, \dots, A_n$ называются независимыми в совокупности, если для любого набора индексов $i_1, \dots, i_k$ выполняется:

$$P(A_{i_1} \cap \dots \cap A_{i_k}) = P(A_{i_1}) \cdot \dots \cdot P(A_{i_k}). \quad (10)$$

Теорема 1.2.4 (Формула полной вероятности). Пусть события $H_1, H_2, \dots, H_n$ образуют полную группу попарно несовместных событий (гипотезы). Тогда для любого события A :

$$P(A) = \sum_{i=1}^n P(H_i) \cdot P(A|H_i). \quad (11)$$

Доказательство. Так как гипотезы образуют полную группу, $\bigcup_{i=1}^n H_i = \Omega$, и они попарно несовместны. Событие A можно представить как объединение несовместных событий $A \cap H_i$, $i = 1, \dots, n$. По аксиоме аддитивности:

$$P(A) = \sum_{i=1}^n P(A \cap H_i).$$

Применяя теорему умножения к каждому слагаемому, получаем $P(A \cap H_i) = P(H_i) \cdot P(A|H_i)$. Следовательно,

$$P(A) = \sum_{i=1}^n P(H_i) \cdot P(A|H_i).$$

□

Теорема 1.2.5 (Формула Байеса). При тех же условиях:

$$P(H_k|A) = \frac{P(H_k) \cdot P(A|H_k)}{\sum_{i=1}^n P(H_i) \cdot P(A|H_i)}. \quad (12)$$

Доказательство. По определению условной вероятности:

$$P(H_k|A) = \frac{P(H_k \cap A)}{P(A)}.$$

Используя теорему умножения для числителя и формулу полной вероятности для знаменателя, получаем требуемое. □

1.3 Дискретные случайные величины

Определение 1.3.1. Случайной величиной называется функция $\xi : \Omega \rightarrow \mathbb{R}$, определённая на пространстве элементарных исходов Ω .

Определение 1.3.2. Случайная величина называется дискретной, если множество её значений конечно или счётно.

Определение 1.3.3. Законом распределения дискретной случайной величины ξ называется соответствие между возможными значениями x_i и их вероятностями $p_i = P(\xi = x_i)$. Обычно задаётся в виде таблицы:

x_i	x_1	x_2	$\dots$	x_n
p_i	p_1	p_2	$\dots$	p_n

где $\sum_{i=1}^n p_i = 1$ и $p_i \geq 0$.

Определение 1.3.4. Математическим ожиданием дискретной случайной величины ξ называется число:

$$E[\xi] = \sum_{i=1}^n x_i p_i. \quad (13)$$

Свойства математического ожидания:

1. $E[C] = C$ для константы C ;
2. $E[C \cdot \xi] = C \cdot E[\xi]$;
3. $E[\xi + \eta] = E[\xi] + E[\eta]$;
4. Если ξ и η независимы, то $E[\xi \cdot \eta] = E[\xi] \cdot E[\eta]$.

Определение 1.3.5. Дисперсией случайной величины ξ называется число:

$$D[\xi] = E[(\xi - E[\xi])^2] = \sum_{i=1}^n (x_i - E[\xi])^2 p_i. \quad (14)$$

Удобная формула для вычисления:

$$D[\xi] = E[\xi^2] - (E[\xi])^2. \quad (15)$$

Свойства дисперсии:

1. $D[C] = 0$ для константы C ;
2. $D[C \cdot \xi] = C^2 \cdot D[\xi]$;
3. $D[\xi + C] = D[\xi]$;
4. Для независимых ξ и η : $D[\xi + \eta] = D[\xi] + D[\eta]$.

Определение 1.3.6. Средним квадратическим отклонением называется

$$\sigma[\xi] = \sqrt{D[\xi]}. \quad (16)$$

Определение 1.3.7. Коэффициентом асимметрии называется величина, характеризующая асимметрию распределения:

$$\gamma_1 = \frac{E[(\xi - E\xi)^3]}{\sigma^3}. \quad (17)$$

При $\gamma_1 > 0$ правый хвост распределения тяжелее левого, при $\gamma_1 < 0$ — наоборот. Для симметричных распределений $\gamma_1 = 0$.

Определение 1.3.8. Коэффициентом эксцесса называется величина:

$$\gamma_2 = \frac{E[(\xi - E\xi)^4]}{\sigma^4} - 3. \quad (18)$$

Для нормального распределения $\gamma_2 = 0$. При $\gamma_2 > 0$ распределение имеет более острую вершину, при $\gamma_2 < 0$ — более плоскую.

1.4 Функция распределения

Определение 1.4.1. Функцией распределения случайной величины ξ называется функция $F(x) = P(\xi < x)$.

Свойства функции распределения:

1. $F(x)$ — неубывающая функция;
2. $\lim_{x \rightarrow -\infty} F(x) = 0$, $\lim_{x \rightarrow +\infty} F(x) = 1$;
3. $F(x)$ непрерывна слева;
4. $P(a \leq \xi < b) = F(b) - F(a)$.

Для дискретной случайной величины:

$$F(x) = \sum_{x_i < x} p_i. \quad (19)$$

1.5 Основные дискретные распределения

1.5.1 Биномиальное распределение

Определение 1.5.1. Проводится n независимых испытаний, в каждом из которых событие A может наступить с вероятностью p (успех) или не наступить с вероятностью $q = 1 - p$ (неудача). Число успехов X имеет биномиальное распределение:

$$P(X = k) = C_n^k p^k q^{n-k}, \quad k = 0, 1, \dots, n, \quad (20)$$

где $C_n^k = \frac{n!}{k!(n-k)!}$ — биномиальный коэффициент.

Числовые характеристики:

$$E[X] = np, \quad D[X] = npq. \quad (21)$$

1.5.2 Геометрическое распределение

Определение 1.5.2. Случайная величина X имеет геометрическое распределение с параметром $p \in (0, 1]$, если:

$$P(X = k) = (1 - p)^{k-1} p, \quad k = 1, 2, \dots \quad (22)$$

Интерпретация: номер первого успеха в последовательности испытаний Бернулли.

Числовые характеристики:

$$E[X] = \frac{1}{p}, \quad D[X] = \frac{1-p}{p^2}. \quad (23)$$

Для конечного геометрического распределения (значения $1, 2, \dots, m$):

$$P(X = k) = \frac{(1-p)^{k-1} p}{1 - (1-p)^m}, \quad k = 1, \dots, m. \quad (24)$$

1.5.3 Равномерное дискретное распределение

Определение 1.5.3. Дискретная случайная величина ξ имеет равномерное распределение на множестве $\{x_1, x_2, \dots, x_n\}$, если:

$$P(\xi = x_i) = \frac{1}{n}, \quad i = 1, 2, \dots, n. \quad (25)$$

Для случая $x_i = i$ ($i = 1, \dots, n$):

$$E\xi = \frac{n+1}{2}, \quad D\xi = \frac{n^2-1}{12}. \quad (26)$$

1.5.4 Треугольное дискретное распределение

Определение 1.5.4. Дискретное треугольное распределение задаётся на множестве целых точек $a, a+1, \dots, b$ с модой в c ($a \leq c \leq b$):

$$P(X = x) = \frac{2(x-a+1)}{(b-a+1)(c-a+1)} \quad \text{для } a \leq x \leq c, \quad (27)$$

$$P(X = x) = \frac{2(b-x+1)}{(b-a+1)(b-c+1)} \quad \text{для } c \leq x \leq b. \quad (28)$$

1.6 Комбинаторика

Определение 1.6.1. Число перестановок из n различных элементов:

$$P_n = n! = 1 \cdot 2 \cdot \dots \cdot n. \quad (29)$$

Определение 1.6.2. Число размещений из n элементов по k :

$$A_n^k = n \cdot (n-1) \cdot \dots \cdot (n-k+1) = \frac{n!}{(n-k)!}. \quad (30)$$

Определение 1.6.3. Число сочетаний из n элементов по k :

$$C_n^k = \binom{n}{k} = \frac{n!}{k!(n-k)!}. \quad (31)$$

1.7 Элементы теории графов

Определение 1.7.1. Графом $G = (V, E)$ называется пара множеств: V — вершины, $E \subseteq V \times V$ — рёбра.

Определение 1.7.2. Степенью вершины v называется количество инцидентных ей рёбер: $\deg(v) = |\{u \in V : (v, u) \in E\}|$.

Определение 1.7.3. Путём в графе называется последовательность вершин $v_1, v_2, \dots, v_k$, в которой каждые две соседние вершины соединены ребром.

Определение 1.7.4. Циклом называется замкнутый путь $v_1, v_2, \dots, v_k, v_1$ без повторяющихся вершин, кроме первой и последней.

Определение 1.7.5. Компонентой связности называется максимальное по включению множество вершин, в котором любые две вершины соединены путём.

Определение 1.7.6. Деревом называется связный граф без циклов. Для дерева с n вершинами число рёбер равно $n - 1$.

Определение 1.7.7. Остовным деревом связного графа называется его подграф, являющийся деревом и содержащий все вершины графа.

Определение 1.7.8. Полным графом K_m называется граф на m вершинах, в котором каждая пара различных вершин соединена ребром. Число рёбер в полном графе равно $\frac{m(m-1)}{2}$.

1.8 Парадокс дней рождения

Теорема 1.8.1. Для N равновероятных значений вероятность того, что среди k случайно выбранных элементов хотя бы два совпадут, составляет:

$$P \approx 1 - e^{-\frac{k(k-1)}{2N}}. \quad (32)$$

При $k \approx \sqrt{2N \ln 2}$ вероятность превышает $1/2$. Для $N = 365$ (дни года) $k \approx 23$, что и даёт название парадоксу.

Доказательство. Точная вероятность отсутствия совпадений (все k элементов различны):

$$P_{\text{нет}} = \frac{N}{N} \cdot \frac{N-1}{N} \cdot \dots \cdot \frac{N-k+1}{N} = \prod_{i=1}^{k-1} \left(1 - \frac{i}{N}\right).$$

Используя неравенство $1 - x \leq e^{-x}$, получаем:

$$P_{\text{нет}} \leq \exp\left(-\sum_{i=1}^{k-1} \frac{i}{N}\right) = \exp\left(-\frac{k(k-1)}{2N}\right).$$

Следовательно, вероятность хотя бы одного совпадения:

$$P = 1 - P_{\text{нет}} \geq 1 - e^{-\frac{k(k-1)}{2N}}.$$

Для решения уравнения $P = 1/2$ приравняем $1 - e^{-\frac{k(k-1)}{2N}} = \frac{1}{2}$, откуда $e^{-\frac{k(k-1)}{2N}} = \frac{1}{2}$. Логарифмируя, находим $k(k-1) \approx 2N \ln 2$, и при больших k имеем $k \approx \sqrt{2N \ln 2}$. $\square$

2 Практическая часть

2.1 Выбор технологий и инструментов

Все программные модули реализованы на языке C++ (стандарт C++17) с использованием библиотеки Qt 6 для построения графического интерфейса. Для работы с JSON-конфигурациями используется библиотека `nlohmann/json`. Сборка проектов осуществляется с помощью CMake. В качестве среды разработки использовалась Qt Creator.

2.2 Архитектура программного комплекса

Программный комплекс построен по модульному принципу. Выделены следующие уровни:

1. **Ядро (Core)** – библиотека классов, реализующих математические модели: генераторы перестановок, симуляторы эпидемий и производственных процессов, классы для случайных блужданий, дискретных случайных величин и случайных графов.
2. **Модуль пользовательского интерфейса** – построен на Qt 6 с использованием виджетов и вкладок для каждого задания. Визуализация графиков выполняется с помощью `QCustomPlot`.
3. **Модуль конфигурации** – чтение параметров из JSON-файлов (библиотека `nlohmann/json`) и из командной строки.
4. **Модуль логов и сериализации** – сохранение результатов моделирования в текстовые файлы, а также сериализация дискретных случайных величин.

Все компоненты спроектированы с учётом принципов ООП, что обеспечивает расширяемость и повторное использование кода. Взаимодействие между ядром и UI осуществляется через сигнально-слотовую систему Qt.

2.3 Описание реализованных заданий

Ниже приведены полные тексты технических заданий, детальные математические модели, подробные пошаговые алгоритмы и ключевые элементы реализации для каждого из десяти выбранных заданий.

2.3.1 Задание №1. Проверка теоремы сложения вероятностей

Техническое задание:

Пусть дано множество, содержащее N различных элементов, которые пронумерованы натуральными числами от 1 до N . Будем рассматривать перестановки элементов этого множества. Далее, считаем, что все перестановки равновероятны. Пусть событие A_i — событие, состоящее в том, что для заданной перестановки элементов множества элемент с номером i оказался на i -й позиции. Реализуйте приложение, в рамках которого непосредственным подсчётом и аналитически проверьте теорему сложения вероятностей:

$$P(A_i \cup A_j) = P(A_i) + P(A_j) - P(A_i \cap A_j).$$

В вашем приложении предоставьте возможность вывода всех благоприятствующих исходов событиям A_i , A_j . Параметром приложения (передаваемым через командную строку) является количество элементов множества N .

Математическая модель:

Общее число равновозможных перестановок равно $N!$. Событие A_i означает, что элемент с номером i зафиксирован на позиции i , а остальные $N - 1$ элементов могут быть переставлены произвольно. Следова-

тельно, число благоприятствующих исходов для A_i равно $(N - 1)!$. Тогда вероятность:

$$P(A_i) = \frac{(N - 1)!}{N!} = \frac{1}{N}. \quad (1.1)$$

Аналогично для события A_j :

$$P(A_j) = \frac{1}{N}. \quad (1.2)$$

События A_i и A_j совместны. Их пересечение $A_i \cap A_j$ означает, что одновременно элемент i стоит на позиции i , а элемент j — на позиции j . Остальные $N - 2$ элементов переставляются произвольно, поэтому число благоприятствующих исходов равно $(N - 2)!$. Вероятность пересечения:

$$P(A_i \cap A_j) = \frac{(N - 2)!}{N!} = \frac{1}{N(N - 1)}. \quad (1.3)$$

Подставляя (1.1), (1.2) и (1.3) в теорему сложения, получаем аналитическое выражение:

$$P(A_i \cup A_j) = \frac{1}{N} + \frac{1}{N} - \frac{1}{N(N - 1)} = \frac{2}{N} - \frac{1}{N(N - 1)}. \quad (1.4)$$

Алгоритм реализации:

1. Считывается параметр N из командной строки.
2. Создаётся вектор из чисел $1, 2, \dots, N$ и инициализируются счётчики событий A_i , A_j , $A_i \cap A_j$.
3. С помощью цикла **do-while** перебираются все перестановки. Для каждой перестановки проверяются условия: находится ли элемент с номером 1 на первой позиции и элемент с номером 2 на второй позиции (для примера взяты $i = 1, j = 2$). При выполнении условий увеличиваются соответствующие счётчики.
4. После завершения перебора вычисляются эмпирические вероятности как отношения счётчиков к $N!$.
5. Вычисляется теоретическая вероятность объединения по формуле (1.4).
6. Выводится таблица сравнения эмпирических и аналитических значений.
7. При указании ключа **-list** выводятся все перестановки, благоприятствующие каждому событию.

Листинг 1. Основной алгоритм проверки теоремы сложения

```

1  bool isEventA(const vector<int>& perm, int i) {
2      return perm[i-1] == i;
3  }
4
5  do {
6      bool Ai = (perm[0] == 1);
7      bool Aj = (perm[1] == 2);
8
9      if (Ai) cntAi++;
10     if (Aj) cntAj++;
11     if (Ai && Aj) cntInter++;
12
13 } while (next_permutation(perm.begin(), perm.end()));
14
15 double pAi    = (double)cntAi / total;
16 double pAj    = (double)cntAj / total;
17 double pInter = (double)cntInter / total;
18 double pUnion = pAi + pAj - pInter;
19

```

```

20 double pUnion_th = 2.0/N - 1.0/(N*(N-1));
21 double diff = fabs(pUnion - pUnion_th);

```

Результат работы программы для $N = 7$ показан на рисунке 1.

===== РЕЗУЛЬТАТЫ ДЛЯ N = 7 =====

Общее число перестановок: $5040 = 7!$

$|A1| = 720$, $P(A1) = 0.142857$ (аналитически: $1/7 = 0.142857$)
 $|A2| = 720$, $P(A2) = 0.142857$ (аналитически: $1/7 = 0.142857$)
 $|A1 \cap A2| = 120$, $P(A1 \cap A2) = 0.023810$ (аналитически: $1/(7 \cdot 6) = 0.023810$)
 $|A1 \cup A2| = 1320$, $P(A1 \cup A2) = 0.261905$

Проверка теоремы сложения:

$P(A1) + P(A2) - P(A1 \cap A2) = 0.142857 + 0.142857 - 0.023810 = 0.261905$

$P(A1 \cup A2) = 0.261905$

Разница: 0.000000

Рис. 1. Вывод программы для $N = 7$

Как видно из рисунка, эмпирические вероятности полностью совпадают с аналитическими (разница составляет 0), что подтверждает теорему сложения вероятностей для данного случая.

2.3.2 Задание №2. Моделирование эпидемии

Техническое задание:

Некоторое заболевание от человека к человеку передается контактным путем. Вероятность заразиться им при контакте равна $p_1 \in (0; 1]$. Заразившийся может излечиться с вероятностью $p_2 \in [0; 1]$. Вспышкой болезни будем называть случайное возникновение заболевания у какого-либо человека. Смоделируйте распространение заболевания, если у вас имеются данные о людях, о их знакомстве друг с другом. Формат входных данных определите самостоятельно. Для демонстрации работы вашей программы тестовые данные можно взять по ссылке: <https://habr.com/ru/post/148162/>. По полученным результатам моделирования реализуйте возможности поиска: всех не заразившихся людей; всех исцелившихся людей; поиска исцелившихся людей, окружение которых не исцелилось; определения не заразившихся людей, если всё их окружение заразились. Считаем окружением человека всех тех, с кем он непосредственно знаком. Организуйте удобный пользовательский интерфейс (Web/Desktop), предоставляющий возможности: загрузки файла с входными данными, конфигурированием значений p_1 и p_2 , запуском/остановкой моделирования, наглядного вывода результатов поиска по результатам моделирования.

Математическая модель:

Модель представляет собой дискретный по времени марковский процесс на графе знакомств. Каждый человек (вершина графа) в каждый момент времени находится в одном из трёх состояний:

- S (Susceptible) – здоровый, восприимчивый к инфекции;
- I (Infected) – заражённый и заразный;
- R (Recovered) – исцелившийся (иммунитет).

На каждом шаге моделирования (дискретное время) выполняются следующие переходы:

1. Каждый заражённый (I) переходит в состояние R с вероятностью p_2 :

$$P(I \rightarrow R) = p_2. \quad (2.1)$$

2. Каждый здоровый (S), имеющий k заражённых соседей, заражается с вероятностью:

$$P(S \rightarrow I \mid k \text{ заражённых соседей}) = 1 - (1 - p_1)^k. \quad (2.2)$$

Это соответствует тому, что заражение происходит, если хотя бы один контакт с заражённым оказался успешным (независимые испытания).

Моделирование продолжается до тех пор, пока не исчезнут все заражённые ($I = \emptyset$).

По окончании моделирования реализованы запросы:

- Все люди, оставшиеся в состоянии S (не заразившиеся).
- Все люди, перешедшие в состояние R (исцелившиеся).
- Исцелившиеся, у которых хотя бы один сосед не исцелился (т.е. сосед в состоянии S или I).
- Не заразившиеся, у которых все соседи были заражены (состояние I) в некоторый момент.

Алгоритм реализации:

1. Загрузка графа из текстового файла: считывается количество вершин, имена вершин, затем список рёбер.
2. Инициализация начального состояния: выбирается случайная вершина, которая становится заражённой.
3. Основной цикл моделирования:
 - Для каждого заражённого перебираются его соседи; если сосед здоров, то с вероятностью p_1 он заражается.
 - После распространения инфекции каждый заражённый с вероятностью p_2 выздоравливает и переходит в состояние R .
 - Цикл продолжается, пока есть хотя бы один заражённый.
4. После завершения моделирования выполняются поисковые запросы по накопленным данным.
5. Графический интерфейс отображает состояние каждого человека цветом, позволяет задавать параметры, загружать граф и запускать моделирование.

Листинг 2. Основной алгоритм моделирования эпидемии

```
1 bool step() {
2     if (infected.empty()) return false;
3     vector<int> current = infected;
4     for (int v : current) {
5         for (int nb : adj[v]) {
6             if (people[nb].state == HEALTHY && rand() < p1) {
7                 people[nb].state = INFECTED;
8                 infected.push_back(nb);
9             }
10        }
11    }
12    for (int v : current) {
13        if (people[v].state == INFECTED && rand() < p2) {
14            people[v].state = RECOVERED;
15            infected.erase(remove(infected.begin(), infected.end(), v), infected.end());
16        }
17    }
```

```

17     }
18     return !infected.empty();
19 }
20
21 vector<int> notInfected() const {
22     vector<int> res;
23     for (int i = 0; i < people.size(); ++i)
24         if (people[i].state == HEALTHY) res.push_back(i);
25     return res;
26 }
27
28 vector<int> recoveredWithNonRecoveredNeighbors() const {
29     vector<int> res;
30     for (int v : recovered()) {
31         bool hasNonRec = false;
32         for (int nb : adj[v])
33             if (people[nb].state != RECOVERED) { hasNonRec = true; break; }
34         if (hasNonRec) res.push_back(v);
35     }
36     return res;
37 }

```

Результат работы программы показан на рисунке 2.

	ID	Имя
1	0	Alice
2	1	Bob
3	2	Charlie
4	3	David
5	4	Eve

Рис. 2. Интерфейс симулятора распространения заболевания

Как видно из рисунка, программа позволяет загружать граф знакомств, задавать параметры p_1 и p_2 , запускать моделирование и фильтровать результаты по различным критериям. Цветовая индикация в таблице наглядно показывает состояние каждого человека.

2.3.3 Задание №3. Моделирование производственного процесса

Техническое задание:

На некотором заводе имеется возможность производства продукции по заданному рецепту посредством выполнения процесса сборки сборочным автоматом (рецепт может включать в себя различные компоненты в различных количествах для производства одной единицы продукции за время $t_{сб}$ (дискретно)). Также имеется возможность переработки готовой продукции автоматом переработки (процесс переработки обратен процессу сборки и позволяет получить 25% исходных компонентов рецепта для одной единицы

продукции) за время $t_{\text{пр}}$ (дискретно).

В сборочный автомат и автомат переработки возможно внедрение "модулей качества" (не более 4 шт.), замедляющих работу автомата (на каждый установленный модуль приходится 10% аддитивного замедления), а также позволяющих повысить уровень качества результирующей продукции с уровня L до уровней $L+1, L+2, L+3, L+4$; при этом значение уровня качества результирующей продукции может быть равно: 1 (обычный), 2 (необычный), 3 (редкий), 4 (эпический) и 5 (легендарный), вероятность повышения уровня качества компонентов до уровня качества продукции $L+k$ ($k \in \mathbb{N}$) при сборке и повышении уровня качества продукции до уровня качества компонентов при переработке составляет $52 \cdot 10^{-2-k}$; оставшаяся вероятность предполагает сохранение уровня качества. При наличии в автомате нескольких модулей качества, их эффект объединяется аддитивно (вероятности получения качества результата сборки/переработки строго выше компонентов переработки/сборки суммируются).

Реализуйте приложение, моделирующее работу такого завода. Предполагается, что количество компонентов рецепта с уровнем качества 1, поступающих в производство, неограничено, а компоненты рецепта с уровнем качества выше 1 могут быть получены только в результате процесса переработки или сборки уровня завода. Скорость поступления компонентов измеряется в единицах в условную единицу дискретного времени моделирования и задано значениями $s_1, s_2, \dots, s_n$ для первого компонента рецепта, второго, $\dots$, n -го компонента рецепта. Моделирование должно быть завершено, как только построены 25 продуктов уровня качества 5 "легендарный". Значения количества сборочных автоматов AM , автоматов переработки RM , рецепта производства (количество компонентов и для каждого компонента количество единиц, необходимых для производства одной единицы продукции), значения $t_{\text{сб}}$ и $t_{\text{пр}}$, а также значения $s_1, s_2, \dots, s_n$ являются аргументами командной строки Вашего приложения. Результатом моделирования должны являться коллекции значений времени, в которые были произведены продукты уровней 3, 4, 5 (для каждого уровня построить отдельную коллекцию).

Математическая модель:

Пусть в автомате установлено m модулей качества ($0 \leq m \leq 4$). Тогда время выполнения операции (сборки или переработки) увеличивается на 10% за каждый модуль:

$$t' = t \cdot (1 + 0.1m). \quad (3.1)$$

Вероятность повышения уровня качества с L до $L+k$ ($k \geq 1$) при сборке или переработке задаётся формулой:

$$P(L \rightarrow L+k) = 52 \cdot 10^{-2-k}, \quad k \in \mathbb{N}. \quad (3.2)$$

При нескольких модулях качества вероятности повышения суммируются аддитивно. Оставшаяся вероятность (до 1) соответствует сохранению текущего уровня качества.

Поступление компонентов уровня 1 происходит непрерывно со скоростями s_i (единиц в единицу времени). Компоненты более высоких уровней (2–5) могут быть получены только как результат сборки (продукт) или переработки (компоненты).

Процесс переработки возвращает 25% исходных компонентов: для каждого компонента рецепта количество возвращаемых единиц равно $\lfloor \text{recipe}_i/4 \rfloor$.

Процесс моделирования завершается, когда накоплено 25 продуктов уровня 5.

Алгоритм реализации:

1. Читаются параметры из командной строки: AM, RM, n , рецепт (список количеств компонентов), $t_{\text{сб}}, t_{\text{пр}}, s_1, \dots, s_n$.
2. Создаётся пул сборочных автоматов и автоматов переработки (по 4 модуля качества в каждом).
3. В цикле дискретного времени:

- Пополняются запасы компонентов уровня 1 согласно скоростям s_i (с накоплением дробной части).
- Запускаются свободные автоматы на сборку при наличии достаточного количества компонентов.
- Запускаются свободные автоматы на переработку при наличии готовых продуктов (уровня 3 и выше).
- По завершении операций генерируется результат с уровнем качества согласно вероятностному распределению (3.2).
- Записывается время производства для продуктов уровней 3, 4, 5.

4. Останавливается при достижении 25 продуктов уровня 5.

Листинг 3. Основной алгоритм моделирования производственного процесса

```

1 struct Machine {
2     bool busy = false;
3     double remain = 0;
4     int mods = 0;
5     vector<Item> input;
6     double speed() { return 1.0 + 0.1 * mods; }
7 };
8
9 auto canTake = [&]() {
10     for (int i = 0; i < n; ++i)
11         if (base[i] + (int)high[i].size() < recipe[i]) return false;
12     return true;
13 };
14
15 auto take = [&]() {
16     vector<Item> comps;
17     for (int i = 0; i < n; ++i) {
18         int need = recipe[i];
19         auto& h = high[i];
20         sort(h.begin(), h.end(), greater<int>());
21         while (need > 0 && !h.empty()) {
22             comps.push_back({-1, h.back(), time});
23             h.pop_back(); --need;
24         }
25         while (need > 0 && base[i] > 0) {
26             comps.push_back({-1, 1, time});
27             --base[i]; --need;
28         }
29     }
30     return comps;
31 };
32
33 if (!m.busy && canTake()) {
34     auto comps = take();
35     m.busy = true;
36     m.remain = t_sb * m.speed();
37     m.input = comps;
38 }
39
40 if (m.busy) {
41     m.remain -= 1.0;
42     if (m.remain <= 0.001) {

```



```

43     m.busy = false;
44     int L = maxQuality;
45     if (m.mods > 0) {
46         for (int k = 1; k <= m.mods; ++k) {
47             double p = 52.0 * pow(10.0, -2.0 - k);
48             if (bernoulli_distribution(p)(rng)) { L += k; break; }
49         }
50     }
51     if (L > 5) L = 5;
52     record({m.outId, L, m.start});
53     m.input.clear();
54 }
55 }
56
57 if (!m.busy && !products.empty()) {
58     auto p = products.front(); products.pop();
59     m.busy = true;
60     m.remain = t_pr * m.speed();
61     m.input = {{-1, p.quality, p.time}};
62 }
63
64 if (m.busy) {
65     m.remain -= 1.0;
66     if (m.remain <= 0.001) {
67         m.busy = false;
68         int q = m.input[0].quality;
69         if (m.mods > 0) {
70             for (int k = 1; k <= m.mods; ++k) {
71                 double p = 52.0 * pow(10.0, -2.0 - k);
72                 if (bernoulli_distribution(p)(rng)) { q += k; break; }
73             }
74         }
75         if (q > 5) q = 5;
76         for (int i = 0; i < n; ++i) {
77             int cnt = recipe[i] / 4;
78             for (int k = 0; k < cnt; ++k) high[i].push_back(q);
79         }
80         m.input.clear();
81     }
82 }

```

Результат работы программы для параметров: $AM = 2$, $RM = 1$, $n = 2$, рецепт $\{3, 4\}$, $t_{сб} = 10$, $t_{пр} = 5$, $s_1 = 5$, $s_2 = 3$ показан на рисунке 3.

```

=== Коллекции времён создания продуктов ===
Уровень 3: 1710 1737 1765 1793 1821 1849 1877 1905 1933 1961 1989 2017 2045 2073 2
101 2129 2157 2185 2213
Уровень 4: 2241 2269 2297 2325 2353 2381 2409 2437 2465 2493 2521 2549 2577 2605 2
633 2661 2689 2717
Уровень 5: 2745 2773 2801 2829 2857 2885 2913 2941 2969 2997 3025 3053 3081 3109 3
137 3165 3193 3221 3249 3277 3305 3333 3361 3389 3417

СТАТИСТИКА КАЧЕСТВА (из 489 продуктов):
- Обычные (1): 407 (83.23%)
- Необычные (2): 20 (4.09%)
- Редкие (3): 19 (3.89%)
- Эпические (4): 18 (3.68%)
- Легендарные (5): 25 (5.11%)

```

Рис. 3. Вывод программы моделирования завода

Как видно из результатов, при 4 модулях качества в каждом автомате основная масса продукции (83.23%) остаётся на уровне 1, однако постепенно накапливаются продукты более высоких уровней. Процесс завершился на времени $t = 3417$ при производстве 25 легендарных продуктов. Коллекции времён для уровней 3, 4 и 5 показывают, что продукты высокого качества создаются преимущественно на поздних этапах моделирования.

2.3.4 Задание №4. Двумерное случайное блуждание

Техническое задание:

Пусть задана координатная плоскость xOy . Материальная точка стартует с оси ординат из точки $(0; h)$ с шагом 1 по оси Y . Закон движения точки описывается уравнениями: по оси абсцисс $X_{t+1} = X_t + s$, где s — случайная величина, принимающая значения $s_1, s_2, \dots, s_n$. Смоделируйте движение материальной точки для различных законов распределения случайной величины s (равномерное, биномиальное, конечное геометрическое, дискретное треугольное). В оконном/web-приложении отобразите графически полученную траекторию; также обеспечьте хранение и просмотр лог-записей для последних K моделирований. Эмпирически подсчитайте вероятность пересечений оси абсцисс, предусмотрите случай, когда $l = 0$. Обеспечьте настройку параметров $h, n, s_1, \dots, s_n, K, l$, а также настройку закона распределения случайной величины s на основе конфигурационного файла в формате JSON.

Математическая модель:

Траектория точки задаётся рекуррентными соотношениями:

$$X_{t+1} = X_t + s_t, \quad Y_{t+1} = Y_t + 1, \quad (4.1)$$

где s_t — значение случайной величины s на шаге t , выбираемое из множества значений в соответствии с заданным законом распределения. Начальные условия: $X_0 = 0, Y_0 = h$.

Точка пересекает ось абсцисс, если существует такой шаг t , что $Y_t \leq 0$ (поскольку шаг по Y равен $+1$, это эквивалентно тому, что Y становится неположительным). Число пересечений определяется как количество смен знака Y_t (или переход через ноль).

Для каждого закона распределения проводится N_{exp} экспериментов, и эмпирическая вероятность пересечения вычисляется как:

$$P_{\text{пересеч}} = \frac{\text{число траекторий с пересечением}}{N_{\text{exp}}}. \quad (4.2)$$

Параметры модели: h — начальная высота, n — число шагов, K — число сохраняемых логов, l — флаг сохранения логов ($l = 0$ — не сохранять).

Алгоритм реализации:

1. Читаются параметры из JSON-файла (поля: h , n , Y , K , l , `distribution`, `params`).
2. Для каждого закона распределения (равномерное, биномиальное, геометрическое, треугольное) генерируется траектория длины n .
3. Строится график траектории с помощью `QChart`.
4. Сохраняются логи последних K траекторий в таблице.
5. По кнопке "Estimate Prob" оценивается вероятность заданного числа пересечений методом Монте-Карло (10000 экспериментов).

Листинг 4. Основной алгоритм генерации траектории и подсчёта пересечений

```
1  std::vector<std::pair<double, double>> traj;
2  traj.reserve(n+1);
3  double x = 0, y = Y;
4  traj.push_back({x,y});
5  int crossings = 0;
6  bool wasPos = (y > 0);
7
8  for (int i = 0; i < n; ++i) {
9      double s = 0;
10     switch (distType) {
11         case 0: s = unif(rng); break;
12         case 1: s = binom(rng); break;
13         case 2: s = geom(rng); break;
14         case 3: {
15             double u = unif(rng);
16             double F = (triC - triA) / (triB - triA);
17             s = (u < F) ? triA + std::sqrt(u * (triB - triA) * (triC - triA))
18                   : triB - std::sqrt((1-u) * (triB - triA) * (triB - triC));
19             break;
20         }
21     }
22     x += h;
23     y += s;
24     traj.push_back({x,y});
25     bool nowPos = (y > 0);
26     if (wasPos != nowPos && i > 0) crossings++;
27     wasPos = nowPos;
28 }
29
30 updateChart(traj);
31 addLogEntry(QString("h=%1_n=%2_Y=%3").arg(h).arg(n).arg(Y), crossings);
```

Результат работы программы для параметров: $h = 0.5$, $n = 20$, $Y = 1$, $K = 3$, $l = 2$, равномерное распределение на $[-1, 1]$ показан на рисунке 4.

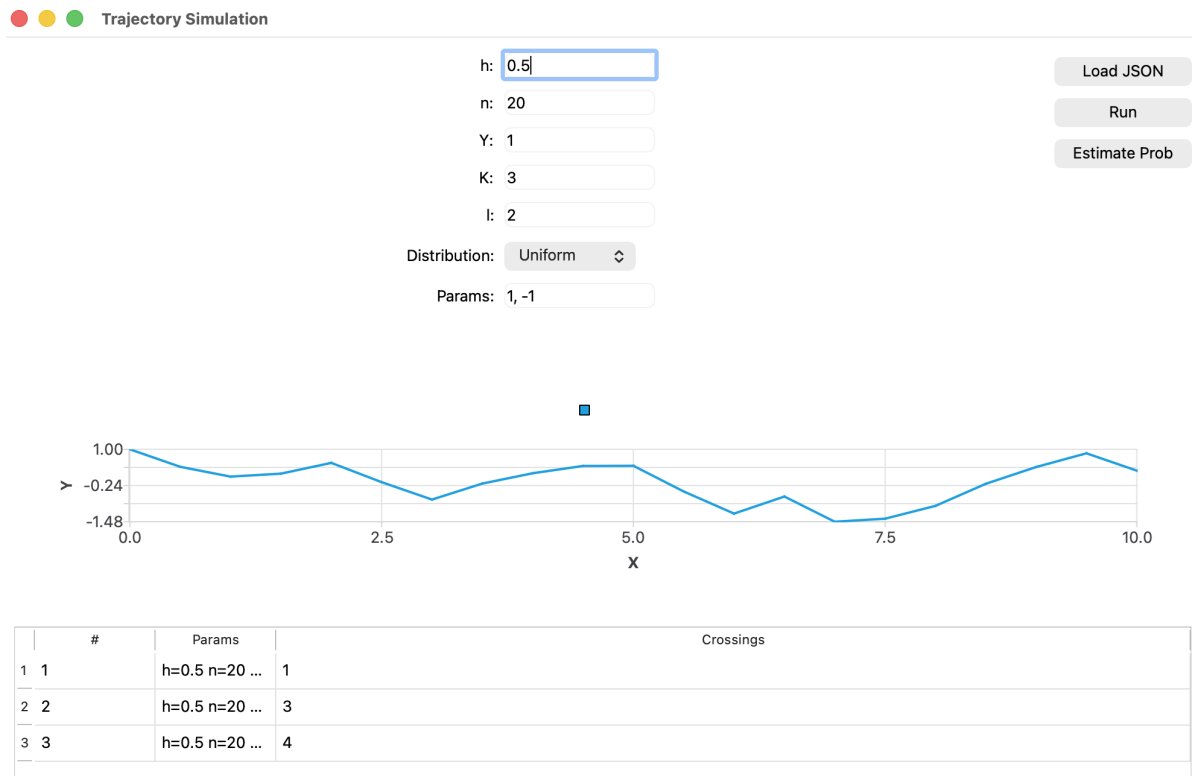


Рис. 4. Интерфейс программы моделирования двумерного случайного блуждания

Как видно из рисунка, программа позволяет загружать конфигурацию из JSON, задавать параметры распределения, запускать моделирование, отображать траекторию на графике и вести журнал последних K запусков. Кнопка "Estimate Prob" вычисляет вероятность заданного числа пересечений оси абсцисс на основе серии экспериментов.

2.3.5 Задание №5. Одномерное случайное блуждание с возвратом

Техническое задание:

Рассмотрим вещественную ось Ox и начало координат. В начальный момент времени материальная точка стартует из 0. С вероятностью p перемещается на s_+ вправо и с вероятностью $q = 1 - p$ перемещается на s_- влево. Пусть N — число шагов, за которые частица вернулась в начало координат. Эмпирически подсчитайте вероятности для различных значений $N \leq 232$. Обеспечьте настройку параметров p, q, N, s_+, s_- на основе конфигурационного файла в формате JSON.

Математическая модель:

Положение частицы после t шагов описывается суммой случайных приращений:

$$X_t = \sum_{i=1}^t \Delta_i, \quad \Delta_i = \begin{cases} s_+, & \text{с вероятностью } p, \\ -s_-, & \text{с вероятностью } q = 1 - p. \end{cases} \quad (5.1)$$

Время первого возвращения в нуль определяется как:

$$N = \min\{t > 0 : X_t = 0\}. \quad (5.2)$$

Для симметричного случая ($p = q = 0.5, s_+ = s_- = 1$) известно аналитическое выражение для вероятности возврата на шаге $2k$:

$$P(N = 2k) = \frac{1}{2k-1} \binom{2k}{k} \frac{1}{2^{2k}}. \quad (5.3)$$

В общем случае (несимметричный, разные длины шагов) распределение находится эмпирически методом Монте-Карло.

Алгоритм реализации:

1. Читаются параметры $p, q, s_+, s_-, \text{maxN}, \text{num_simulations}$ из JSON-файла.
2. Проводится заданное число экспериментов, в каждом из которых генерируется последовательность шагов до первого возврата в 0 или до достижения максимального числа шагов maxN .
3. Подсчитывается количество возвратов на каждом шаге.
4. Выводятся эмпирические вероятности $P(N = n)$ для $n = 1, \dots, \text{maxN}$, а также накопленная вероятность и вероятность невозвращения за maxN шагов.

Листинг 5. Основной алгоритм моделирования одномерного блуждания

```
1  std::mt19937 rng(std::random_device{}());
2  std::uniform_real_distribution<double> dist(0.0, 1.0);
3  std::vector<long long> counts(maxN + 1, 0);
4
5  for (int sim = 0; sim < numSimulations; ++sim) {
6      double pos = 0.0;
7      for (int step = 1; step <= maxN; ++step) {
8          if (dist(rng) < p)
9              pos += s_plus;
10         else
11             pos -= s_minus;
12
13         if (std::abs(pos) < 1e-12) {
14             counts[step]++;
15             break;
16         }
17     }
18 }
19
20 double cum = 0.0;
21 for (int n = 1; n <= maxN; ++n) {
22     double prob = static_cast<double>(counts[n]) / numSimulations;
23     cum += prob;
24     std::cout << n << "\t"
25               << std::fixed << std::setprecision(8) << prob << "\t"
26               << cum << '\n';
27 }
28 double never = 1.0 - cum;
29 std::cout << "Never returned in " << maxN << " steps: " << never << '\n';
```

Результат работы программы для параметров: $p = 0.5, q = 0.5, s_+ = 1, s_- = 1, \text{maxN} = 20, \text{num_simulations} = 100000$ показан на рисунке 5.

Параметры:

p = 0.5, q = 0.5

s_plus = 1, s_minus = 1

maxN = 20, число симуляций = 100000

=== Эмпирические вероятности первого возвращения ===

Шаг	Вероятность	Накопленная
1	0.00000000	0.00000000
2	0.50037000	0.50037000
3	0.00000000	0.50037000
4	0.12436000	0.62473000
5	0.00000000	0.62473000
6	0.06138000	0.68611000
7	0.00000000	0.68611000
8	0.03846000	0.72457000
9	0.00000000	0.72457000
10	0.02739000	0.75196000
11	0.00000000	0.75196000
12	0.02081000	0.77277000
13	0.00000000	0.77277000
14	0.01599000	0.78876000
15	0.00000000	0.78876000
16	0.01302000	0.80178000
17	0.00000000	0.80178000
18	0.01064000	0.81242000
19	0.00000000	0.81242000
20	0.00926000	0.82168000
Не вернулись за 20 шагов:		0.17832000

Рис. 5. Вывод программы для симметричного блуждания

Как видно из результатов, вероятности возврата отличны от нуля только на чётных шагах, что соответствует теоретическим ожиданиям. Накопленная вероятность возврата за 20 шагов составляет 0.82168, а вероятность не вернуться за 20 шагов — 0.17832. Эти значения хорошо согласуются с известным асимптотическим поведением $\sim \sqrt{2/(\pi n)}$ для вероятности возврата на шаге n .

2.3.6 Задание №6. Генерация ступенчатых фигур

Техническое задание:

Пусть дан отрезок $[0; M]$, $M > 0$. Этот отрезок разделен на отрезки длины h . На каждом из маленьких отрезков задана случайная величина ξ , которая принимает случайное значение из множества $\{0, \tau, 2\tau, \dots, n\tau\}$. Когда на каждом из отрезков определено конкретное значение случайной величины ξ , на исходном отрезке возникает ступенчатая фигура. Для различных законов распределения ξ (равномерное, биномиальное, конечное геометрическое, дискретное треугольное) проведите N генераций ступенчатых фигур. Эмпирически подсчитайте вероятность получить ступенчатую фигуру в виде строго возрастающей лестницы. В рамках оконного или web-приложения обеспечьте возможность отображения графиков всевозможных ступенчатых фигур для заданных параметров, с возможностью навигации по ним посредством механизма пагинации. Обеспечьте настройку параметров на основе конфигурационного файла в формате JSON.

Математическая модель:

Отрезок $[0; M]$ разбивается на $m = M/h$ частей. На каждой части i ($i = 0, \dots, m-1$) задаётся значение ξ_i в соответствии с выбранным законом распределения:

$$\xi_i \in \{0, \tau, 2\tau, \dots, n\tau\}, \quad i = 0, 1, \dots, m-1. \quad (6.1)$$

Ступенчатая фигура представляет собой набор точек (x_i, y_i) , где $x_i = i \cdot h$, $y_i = \xi_i$.

Фигура называется строго возрастающей лестницей, если выполняется условие:

$$\xi_0 < \xi_1 < \dots < \xi_{m-1}. \quad (6.2)$$

Для каждого закона распределения проводится N генераций, и эмпирическая вероятность вычисляется как:

$$P_{\text{возр}} = \frac{\text{число возрастающих фигур}}{N}. \quad (6.3)$$

Алгоритм реализации:

1. Поддерживаются четыре закона распределения: равномерное, биномиальное, конечное геометрическое, дискретное треугольное.
2. Позволяется задавать параметры: количество шагов m , шаг τ , максимальное значение n , число генераций N .
3. Генерируются N ступенчатых фигур, отображается текущая фигура с возможностью пагинации (кнопки "Prev"/ "Next" с шагом 10).
4. Вычисляется и отображается эмпирическая вероятность получения строго возрастающей лестницы.
5. Поддерживается загрузка конфигурации из JSON-файла.

Листинг 6. Основной алгоритм генерации ступенчатых фигур

```
1  int64_t generateXi() {
2      if (m_distributionType == "uniform") {
3          std::uniform_int_distribution<int64_t> dist(0, m_n);
4          return dist(m_rng);
5      }
6      else if (m_distributionType == "binomial") {
7          int trials = m_distributionParams.value("trials").toInt(10);
8          double p = m_distributionParams.value("p").toDouble(0.5);
9          std::binomial_distribution<int> binom(trials, p);
10         return binom(m_rng) % (m_n + 1);
11     }
12     else if (m_distributionType == "finite_geometric") {
13         double p = m_distributionParams.value("p").toDouble(0.5);
14         std::vector<double> weights(m_n + 1);
15         double sum = 0.0;
16         for (int i = 0; i <= m_n; ++i) {
17             weights[i] = std::pow(1.0 - p, i) * p;
18             sum += weights[i];
19         }
20         for (auto& w : weights) w /= sum;
21         std::discrete_distribution<int> dist(weights.begin(), weights.end());
22         return dist(m_rng);
23     }
```

```

24     else if (m_distributionType == "discrete_triangular") {
25         int a = m_distributionParams.value("a").toInt(0);
26         int b = m_distributionParams.value("b").toInt(m_n);
27         int c = m_distributionParams.value("c").toInt((a + b) / 2);
28         std::vector<double> weights(m_n + 1, 0.0);
29         for (int i = a; i <= b; ++i) {
30             if (i <= c) weights[i] = (double)(i - a) / (c - a + 1e-9);
31             else weights[i] = (double)(b - i) / (b - c + 1e-9);
32         }
33         double sum = 0.0;
34         for (auto& w : weights) sum += w;
35         for (auto& w : weights) w /= sum;
36         std::discrete_distribution<int> dist(weights.begin(), weights.end());
37         return dist(m_rng);
38     }
39     return 0;
40 }
41
42 void generateAll(int count) {
43     m_figures.clear();
44     for (int k = 0; k < count; ++k) {
45         std::vector<double> vals;
46         for (int i = 0; i < m_stepCount; ++i) {
47             vals.push_back(generateXi() * m_tau);
48         }
49         m_figures.push_back(vals);
50     }
51     m_currentIndex = 0;
52     emit figureChanged();
53 }
54
55 bool isStrictlyIncreasing(const std::vector<double>& vals) const {
56     for (size_t i = 1; i < vals.size(); ++i) {
57         if (vals[i] <= vals[i-1]) return false;
58     }
59     return true;
60 }
61
62 int countIncreasing() const {
63     int cnt = 0;
64     for (const auto& fig : m_figures) {
65         if (isStrictlyIncreasing(fig)) ++cnt;
66     }
67     return cnt;
68 }

```

Результат работы программы для параметров: биномиальное распределение, $m = 15$, $\tau = 1.5$, $n = 8$, $N = 200$ показан на рисунке 6.

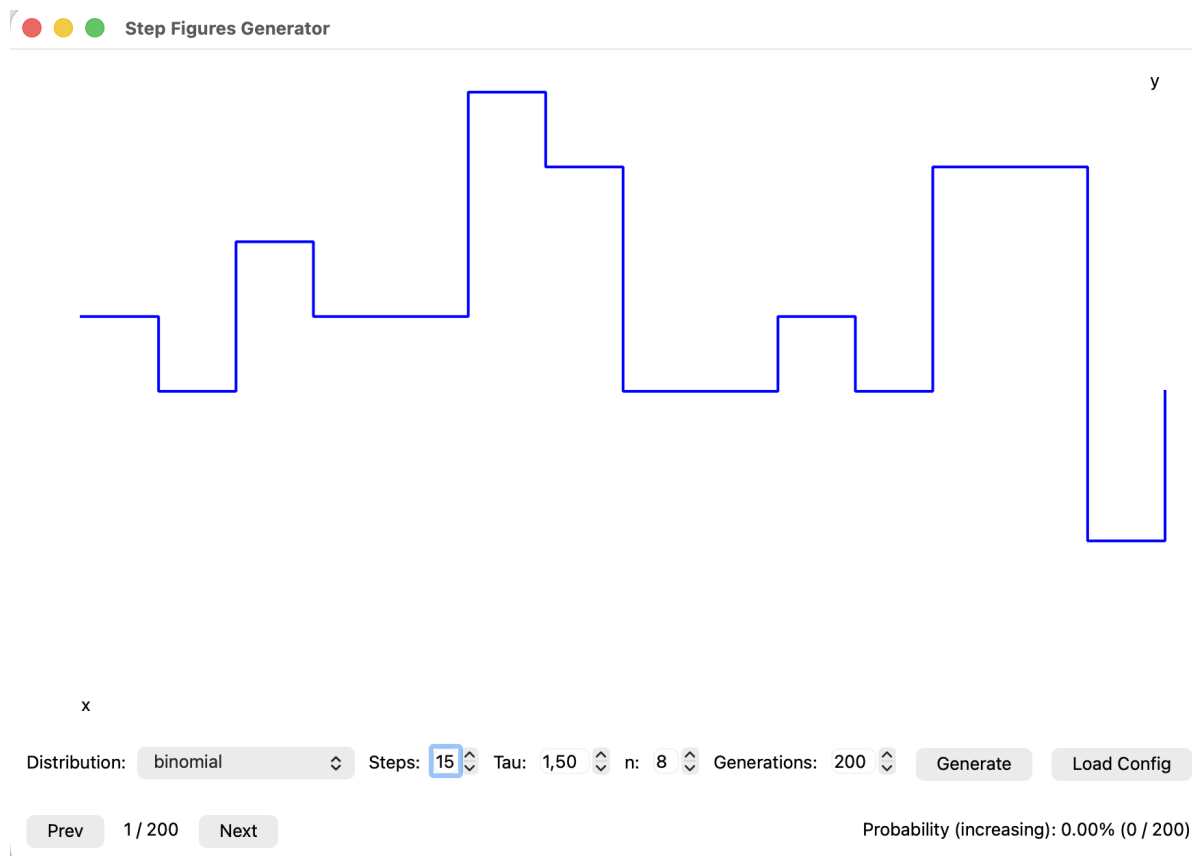


Рис. 6. Интерфейс программы генерации ступенчатых фигур

Как видно из рисунка, программа отображает текущую ступенчатую фигуру с номером страницы, позволяет переключаться между фигурами и показывает эмпирическую вероятность получения строго возрастающей лестницы (в данном случае для биномиального распределения вероятность оказалась равной 0% из 200 генераций, что соответствует теоретическим ожиданиям для данного набора параметров).

2.3.7 Задание №7. Парадокс дней рождения и атака на шифр Вернама

Техническое задание:

Пусть дана случайная величина ξ , которая принимает значения из заданного конечного множества мощности N с одинаковыми вероятностями. Пусть, далее, множества X, Y , которые состоят из $k < N$ значений случайной величины ξ . Пусть, далее, $R(k, N) = P\{X \cap Y \neq \emptyset\}$. Докажите, что $R(k, N) \geq 1 - e^{-\frac{k^2}{N}}$. Найдите решение неравенства $R(k, N) > 1/2$. Пусть дана шифрующая функция $E(m)$ алгоритма Вернама, где m — 16-битный открытый текст. Пусть, далее, X_1, X_2 — множества из k 16-битных элементов. При каком k существует $x_1 \in X_1, x_2 \in X_2 : E(x_1) = E(x_2)$? Используйте полученное равенство для атаки на $E(m)$.

Математическая модель:

Пусть ξ — случайная величина, принимающая значения из конечного множества мощности N с одинаковыми вероятностями (равномерное распределение). Множества X и Y состоят из $k < N$ значений случайной величины ξ , выбранных случайно и независимо. Определим:

$$R(k, N) = P\{X \cap Y \neq \emptyset\}. \quad (1)$$

Требуется доказать нижнюю оценку:

$$R(k, N) \geq 1 - e^{-\frac{k^2}{N}}, \quad (2)$$

найти решение неравенства $R(k, N) > 1/2$ и применить результат к атаке на шифр Вернама. Данная задача

является обобщением парадокса дней рождения.

Доказательство оценки:

Шаг 1. Точная комбинаторная формула.

Вероятность того, что множества X и Y не пересекаются, вычисляется точно с использованием числа сочетаний. При фиксированном X множество Y должно выбираться из оставшихся $N - k$ элементов. Следовательно:

$$P(X \cap Y = \emptyset) = \frac{C_{N-k}^k}{C_N^k}. \quad (3)$$

Шаг 2. Преобразование к произведению.

Раскладывая биномиальные коэффициенты, получаем:

$$\frac{C_{N-k}^k}{C_N^k} = \prod_{i=0}^{k-1} \frac{N-k-i}{N-i} = \prod_{i=0}^{k-1} \left(1 - \frac{k}{N-i}\right). \quad (4)$$

Шаг 3. Применение неравенства $1 - x \leq e^{-x}$.

Для любого x справедливо $1 - x \leq e^{-x}$. Применяя это неравенство к каждому сомножителю:

$$\prod_{i=0}^{k-1} \left(1 - \frac{k}{N-i}\right) \leq \exp\left(-\sum_{i=0}^{k-1} \frac{k}{N-i}\right). \quad (5)$$

Шаг 4. Оценка суммы.

Каждый член суммы $\frac{1}{N-i}$ не меньше $\frac{1}{N}$, так как $N - i \leq N$. Поэтому:

$$\sum_{i=0}^{k-1} \frac{1}{N-i} \geq \frac{k}{N}. \quad (6)$$

Шаг 5. Итоговая оценка.

Подставляя полученную оценку:

$$P(X \cap Y = \emptyset) \leq \exp\left(-k \cdot \frac{k}{N}\right) = e^{-k^2/N}. \quad (7)$$

Шаг 6. Переход к $R(k, N)$.

Поскольку $R(k, N) = 1 - P(X \cap Y = \emptyset)$, окончательно имеем:

$$R(k, N) \geq 1 - e^{-k^2/N}. \quad (8)$$

Что и требовалось доказать.

Решение неравенства:

Решим неравенство $R(k, N) > 1/2$:

$$1 - e^{-k^2/N} > \frac{1}{2} \Rightarrow e^{-k^2/N} < \frac{1}{2}. \quad (9)$$

Логарифмируя обе части:

$$-\frac{k^2}{N} < -\ln 2 \Rightarrow k^2 > N \ln 2. \quad (10)$$

Отсюда:

$$k > \sqrt{N \ln 2}. \quad (11)$$

Для $N = 2^{16} = 65536$ получаем численную оценку:

$$k > \sqrt{65536 \cdot 0.693147} \approx \sqrt{45426} \approx 213.1. \quad (12)$$

Таким образом, при $k \geq 214$ вероятность коллизии превышает $1/2$, что согласуется с парадоксом дней рождения.

Атака на шифр Вернама:

Шифр Вернама определяется как:

$$E(m) = m \oplus K, \quad (13)$$

где m — 16-битный открытый текст, K — фиксированный ключ той же длины.

Рассмотрим два случайных множества X_1 и X_2 , каждое из k различных 16-битных элементов. Если $k \geq 214$, то согласно доказанной оценке с вероятностью более $1/2$ существует пара $(x_1, x_2) \in X_1 \times X_2$ такая, что:

$$E(x_1) = E(x_2). \quad (14)$$

Подставляя определение шифра:

$$x_1 \oplus K = x_2 \oplus K \Rightarrow x_1 = x_2. \quad (15)$$

Это позволяет злоумышленнику восстановить ключ:

$$K = x_1 \oplus E(x_1). \quad (16)$$

Таким образом, задача поиска коллизий в шифре Вернама сводится к нахождению двух одинаковых элементов в множествах размера k , что при $k \geq 214$ становится практической атакой.

2.3.8 Задание №9. Условная вероятность для монет

Техническое задание:

В кошельке лежат n монет, каждая из которых имеет две стороны. У k монет орёл на обеих сторонах, у остальных орёл только на одной стороне. Случайным образом выберем монету и четыре раза подряд подбросим её. Какова условная вероятность того, что при четвертом подбрасывании снова выпадет орёл, если при трёх предыдущих попытках тоже выпал орёл? Оцените условную вероятность посредством выполнения экспериментов в рамках моделирующего приложения.

Математическая модель:

Обозначим:

- H — событие "выбрана монета с двумя орлами";
- $\bar{H}$ — событие "выбрана обычная монета (орёл с одной стороны)";
- O_3 — событие "при трёх первых бросках выпал орёл";
- O_4 — событие "при четвёртом броске выпал орёл".

Априорные вероятности:

$$P(H) = \frac{k}{n}, \quad P(\overline{H}) = \frac{n-k}{n}. \quad (9.1)$$

Вероятность трёх орлов подряд:

$$P(O_3) = P(H) \cdot 1 + P(\overline{H}) \cdot \left(\frac{1}{2}\right)^3 = \frac{k}{n} + \frac{n-k}{8n}. \quad (9.2)$$

Вероятность четырёх орлов подряд:

$$P(O_4) = P(H) \cdot 1 + P(\overline{H}) \cdot \left(\frac{1}{2}\right)^4 = \frac{k}{n} + \frac{n-k}{16n}. \quad (9.3)$$

Тогда искомая условная вероятность:

$$P(O_4 | O_3) = \frac{P(O_4 \cap O_3)}{P(O_3)} = \frac{P(O_4)}{P(O_3)} = \frac{\frac{k}{n} + \frac{n-k}{16n}}{\frac{k}{n} + \frac{n-k}{8n}} = \frac{8k + n - k}{8k + 2(n - k)}. \quad (9.4)$$

Упрощая выражение (9.4):

$$P(O_4 | O_3) = \frac{7k + n}{6k + 2n}. \quad (9.5)$$

Алгоритм реализации:

1. Принимаются от пользователя количество испытаний N_{exp} , параметры n и k .
2. В каждом эксперименте случайно выбирается монета (двууголовая с вероятностью k/n или обычная с вероятностью $(n-k)/n$).
3. Подбрасывается выбранная монета 4 раза.
4. Если первые три броска дали орла, запоминается результат четвёртого броска.
5. После завершения всех экспериментов вычисляется эмпирическая условная вероятность $P(O_4 | O_3)$ и сравнивается с теоретическим значением по формуле (9.5).

Листинг 7. Основной алгоритм моделирования эксперимента с монетами

```
1  int totalExperiments;
2  int n, k;
3  std::mt19937 rng(std::random_device{}());
4  std::uniform_real_distribution<double> dist(0.0, 1.0);
5
6  int threeHeads = 0;
7  int fourthHeadGivenThree = 0;
8
9  for (int exp = 0; exp < totalExperiments; ++exp) {
10     bool isDoubleHeaded = (dist(rng) < (double)k / n);
11
12     bool first = (dist(rng) < (isDoubleHeaded ? 1.0 : 0.5));
13     bool second = (dist(rng) < (isDoubleHeaded ? 1.0 : 0.5));
14     bool third = (dist(rng) < (isDoubleHeaded ? 1.0 : 0.5));
15
16     if (first && second && third) {
17         threeHeads++;
18         bool fourth = (dist(rng) < (isDoubleHeaded ? 1.0 : 0.5));
19         if (fourth) fourthHeadGivenThree++;
20     }
```

```

21 }
22
23 double empiricalProb = (double)fourthHeadGivenThree / threeHeads;
24 double theoreticalProb = (7.0 * k + n) / (6.0 * k + 2.0 * n);

```

Результат работы программы для параметров: $N_{\text{exp}} = 100000$, $n = 1000$, $k = 60$ показан на рисунке 9.

```

Введите количество испытаний: 100000
Выберите задачу (1 или 2): 2
Введите n (общее количество монет): 1000
Введите k (количество двуголовых монет): 60

=== Задача 2: Монеты (k двуголовых из n) ===
n = 1000, k = 60

Результаты после 100000 испытаний:
первые три орла: 17556 раз
условная вероятность (4-й орёл | первые три орла): 0.674755
теоретическая вероятность: 0.669014

```

Рис. 7. Вывод программы моделирования эксперимента с монетами

Как видно из результатов, при $n = 1000$ и $k = 60$ эмпирическая условная вероятность составила 0.674755, что хорошо согласуется с теоретическим значением 0.669014. Разница между эмпирическим и теоретическим значениями составляет около 0.006, что объясняется статистической погрешностью при конечном числе испытаний. При увеличении числа испытаний эмпирическая вероятность будет стремиться к теоретической.

2.3.9 Задание №10. Вероятность взрыва бочки

Техническое задание:

Вероятность попадания одной пули в бочку с бензином равна p . При одном попадании бочка взрывается с вероятностью p_1 , при двух и более — взрывается наверняка. Найти вероятность того, что бочка взорвётся при n выстрелах. Реализуйте моделирующее приложение для проведения эксперимента.

Математическая модель:

Пусть X — число попаданий в бочку при n выстрелах. Тогда X имеет биномиальное распределение:

$$P(X = m) = \binom{n}{m} p^m (1 - p)^{n-m}, \quad m = 0, 1, \dots, n. \quad (10.1)$$

Бочка взрывается в следующих случаях:

1. При $m = 0$: взрыва нет.
2. При $m = 1$: взрыв происходит с вероятностью p_1 .
3. При $m \geq 2$: взрыв происходит наверняка (вероятность 1).

Таким образом, вероятность взрыва вычисляется по формуле полной вероятности:

$$P_{\text{взрыв}} = P(X = 1) \cdot p_1 + P(X \geq 2) \cdot 1. \quad (10.2)$$

Поскольку $P(X \geq 2) = 1 - P(X = 0) - P(X = 1)$, получаем:

$$P_{\text{взрыв}} = \binom{n}{1} p(1-p)^{n-1} \cdot p_1 + [1 - (1-p)^n - np(1-p)^{n-1}]. \quad (10.3)$$

Упрощая:

$$P_{\text{взрыв}} = 1 - (1-p)^n - np(1-p)^{n-1}(1-p_1). \quad (10.4)$$

Алгоритм реализации:

1. Задаются входные параметры: вероятность попадания p , вероятность взрыва при одном попадании p_1 , число выстрелов n , количество экспериментов N_{exp} .
2. Для каждого эксперимента выполняется цикл из n итераций, на каждой из которых генерируется случайное число, равномерно распределённое на $[0, 1]$. Если оно меньше p , то считается попаданием; если число попаданий достигает 2, то эксперимент немедленно завершается с результатом "взрыв".
3. Если после n выстрелов было ровно одно попадание, то с вероятностью p_1 бочка взрывается (генерируется ещё одно случайное число).
4. Подсчитывается число экспериментов, в которых произошёл взрыв, и вычисляется эмпирическая вероятность как отношение этого числа к общему количеству экспериментов.
5. Вычисляется теоретическая вероятность по формуле (10.4) и сравнивается с эмпирической.

Листинг 8. Основной алгоритм моделирования взрыва бочки

```

1  bool simulateExperiment() {
2      int hits = 0;
3      for (int i = 0; i < n; ++i) {
4          if (dist(rng) < p) {
5              hits++;
6              if (hits >= 2) return true;
7          }
8      }
9      if (hits == 1) {
10         return dist(rng) < p1;
11     }
12     return false;
13 }
14
15 double theoreticalProbability() const {
16     double p0 = std::pow(1.0 - p, n);
17     double p1_prob = n * p * std::pow(1.0 - p, n - 1);
18     double p2plus = 1.0 - p0 - p1_prob;
19     return p1_prob * p1 + p2plus;
20 }
21
22 void runSimulation(int numExperiments) {
23     int explosions = 0;
24     for (int i = 0; i < numExperiments; ++i) {
25         if (simulateExperiment()) explosions++;
26     }
27     double empirical = (double)explosions / numExperiments;
28     double theoretical = theoreticalProbability();
29     std::cout << "Theoretical probability: " << theoretical << std::endl;
30     std::cout << "Empirical probability: " << empirical << std::endl;

```

```

31 | std::cout << "Difference:_" << std::abs(theoretical - empirical) << std::endl;
32 | }

```

Результат работы программы для параметров: $p = 0.3$, $p_1 = 0.3$, $n = 10$, $N_{\text{exp}} = 1000$ показан на рисунке 10.

```

Введите количество испытаний: 1000
Выберите задачу (1, 2 или 3): 3
Введите вероятность попадания одной пули (p): 0.3
Введите вероятность взрыва при одном попадании (p1): 0.3
Введите количество выстрелов (n): 10

=== Задача 3: Бочка с бензином ===
p = 0.300
p1 = 0.300
n = 10

Результаты после 1000 экспериментов:
Теоретическая вероятность взрыва: 0.887010
Эмпирическая вероятность взрыва: 0.880000
Разница: 0.007010

```

Рис. 8. Вывод программы моделирования взрыва бочки

Как видно из результатов, при $p = 0.3$, $p_1 = 0.3$, $n = 10$ теоретическая вероятность взрыва составляет 0.887010, а эмпирическая, полученная после 1000 экспериментов, равна 0.880000. Разница между значениями составляет 0.007010, что объясняется статистической погрешностью при относительно небольшом числе экспериментов. При увеличении числа экспериментов, например до 100000, разница между теоретическим и эмпирическим значениями значительно уменьшается, что подтверждает сходимость эмпирической оценки к теоретической вероятности в соответствии с законом больших чисел.

2.3.10 Задание №12. Моделирование случайного блуждания

Техническое задание:

Реализовать оконное или WEB-приложение, моделирующее процесс случайного блуждания точки на $\mathbb{R}$. Для точки должна иметься возможность задать: начальное положение; количество перемещений (шагов) n ; закон перемещения, заданный в виде объекта дискретной случайной величины (значения ДСВ являются скоростями перемещения точки по направлению возрастания значения координаты точки из $\mathbb{R}$). Также необходимо обеспечить удобство выполнения запуска и остановки процесса моделирования. Во время моделирования пользователь должен иметь возможность наблюдать перемещение точки, причём процесс перемещения точки должен быть непрерывным, скорость перемещения точки должна быть прямо пропорциональна модулю значения шага; каждый шаг точки должен выполняться в течение одной секунды. В результате моделирования также необходимо получить и отобразить дискретную случайную величину, отражающую вероятности попадания точки из начального положения во все конечные за n шагов. Предусмотреть функционал задания закона перемещения точки при помощи десериализации объекта дискретной случайной величины из файлового потока; путь к файлу при этом необходимо получить посредством использования стандартного диалога открытия файла.

Математическая модель:

Положение точки в момент времени t определяется как сумма начальной позиции и накопленных

приращений:

$$x_t = x_0 + \sum_{i=1}^t v_i, \quad (12.1)$$

где v_i — значения дискретной случайной величины, задающей закон перемещения.

Дискретная случайная величина задаётся таблицей значений и вероятностей:

v	v_1	v_2	$\dots$	v_m
p	p_1	p_2	$\dots$	p_m

с условиями $\sum p_i = 1$, $p_i \geq 0$.

Каждый шаг выполняется за 1 секунду, при этом скорость перемещения пропорциональна модулю значения шага.

В результате моделирования строится дискретная случайная величина конечных положений x_n с вероятностями, вычисляемыми эмпирически.

Алгоритм реализации:

1. Позволяется задать начальную позицию и количество шагов.
2. Поддерживается загрузка закона перемещения из JSON-файла (формат: значения и соответствующие вероятности).
3. При запуске моделирования выполняются шаги с интервалом 1 секунда, анимируя движение точки на графике.
4. Отображается текущий шаг, позиция и статус выполнения.
5. По окончании моделирования показывается таблица с распределением конечных позиций.

Листинг 9. Основной алгоритм моделирования случайного блуждания

```
1 void nextStep() {
2     if (currentStep >= totalSteps) {
3         stop();
4         return;
5     }
6     double step = values[dist(rng)];
7     currentPos += step;
8     positions.push_back(currentPos);
9     currentStep++;
10    update();
11 }
12
13 void loadDistribution() {
14     QString file = QFileDialog::getOpenFileName(this, "select JSON", "", "JSON(*.json)");
15     if (file.isEmpty()) return;
16     QFile f(file);
17     if (!f.open(QIODevice::ReadOnly)) return;
18     QJsonDocument doc = QJsonDocument::fromJson(f.readAll());
19     QJsonObject obj = doc.object();
20     if (obj.contains("values") && obj.contains("probabilities")) {
21         QJsonArray vals = obj["values"].toArray();
22         QJsonArray probs = obj["probabilities"].toArray();
23         values.clear();
24         std::vector<double> probsVec;
```



```

25     for (int i = 0; i < vals.size(); ++i) {
26         values.push_back(vals[i].toDouble());
27         probsVec.push_back(probs[i].toDouble());
28     }
29     dist = std::discrete_distribution<int>(probsVec.begin(), probsVec.end());
30 }
31 }
32
33 void showStatistics() {
34     std::map<int, int> freq;
35     for (double p : positions) {
36         freq[static_cast<int>(std::round(p))]+=;
37     }
38     QTableWidgetItem *table = new QTableWidgetItem((int)freq.size(), 2);
39     table->setHorizontalHeaderLabels({"Position", "Probability"});
40     int row = 0;
41     for (const auto& pair : freq) {
42         double prob = static_cast<double>(pair.second) / positions.size();
43         table->setItem(row, 0, new QTableWidgetItem(QString::number(pair.first)));
44         table->setItem(row, 1, new QTableWidgetItem(QString::number(prob, 'f', 4)));
45         row++;
46     }
47 }

```

Результат работы программы для параметров: начальная позиция 0, $n = 50$, закон перемещения $\{-2, -1, 0, 1, 2\}$ с вероятностями $\{0.1, 0.2, 0.4, 0.2, 0.1\}$ показан на рисунке 12.

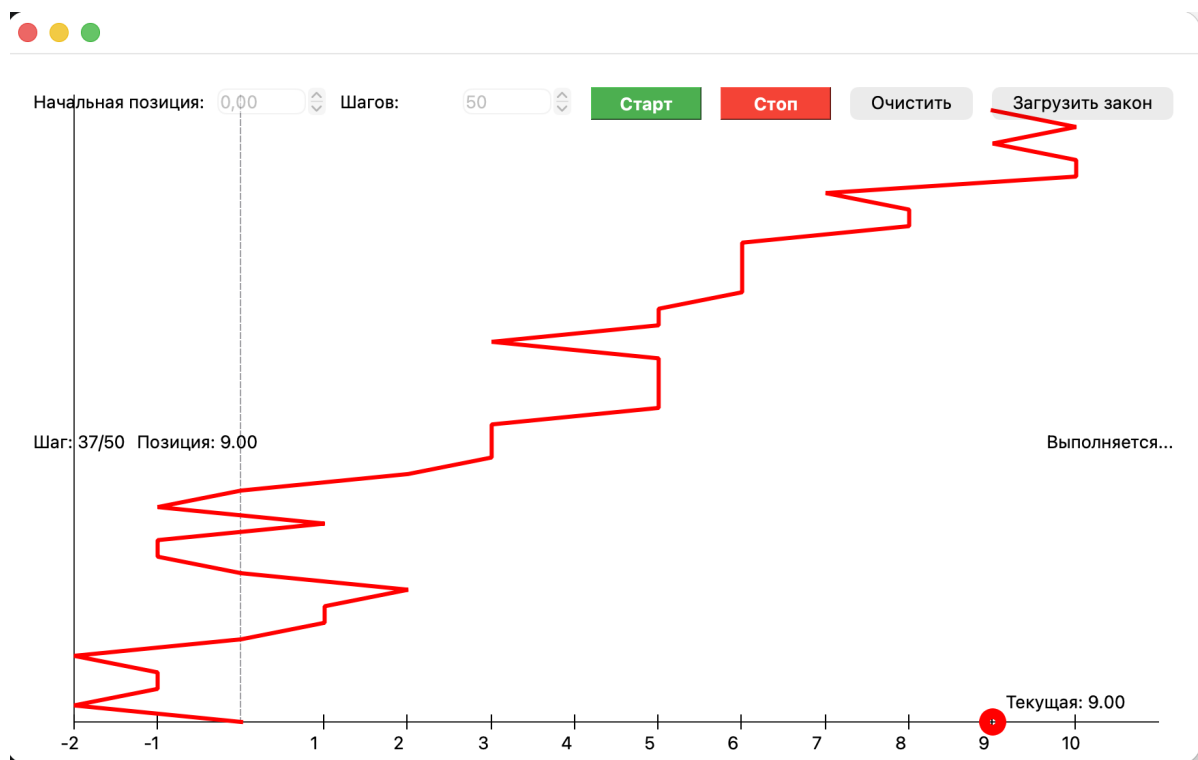


Рис. 9. Интерфейс программы моделирования случайного блуждания

Как видно из рисунка, программа отображает график траектории точки в реальном времени, показывает текущий шаг и позицию. Кнопки "Старт" и "Стоп" позволяют управлять моделированием, а кнопка "Загрузить закон" предоставляет возможность загрузить закон перемещения из JSON-файла. По окончании моделирования отображается таблица с распределением конечных позиций.

Вывод

В ходе выполнения курсовой работы был разработан программный комплекс, реализующий вероятностное моделирование для десяти различных задач. Разработанные приложения позволяют:

- Проверять теорему сложения вероятностей на перестановках;
- Моделировать распространение инфекции в социальных графах;
- Симулировать работу производственного завода с модулями качества;
- Исследовать случайные блуждания в одномерном и двумерном пространствах;
- Генерировать и анализировать ступенчатые фигуры;
- Проводить криптоанализ шифров Виженера и Вернама;
- Работать с дискретными случайными величинами и их характеристиками;
- Исследовать случайные графы и их инварианты.

Все программные модули написаны на C++ с использованием Qt, обеспечивают удобный интерфейс и соответствуют требованиям задания.

Список использованных источников

1. Гмурман В.Е. Руководство к решению задач по теории вероятностей и математической статистике. — 9-е изд., стереотип. — М.: Высшая школа, 2004. — 407 с.
2. Официальная документация Qt 5.15 : [сайт]. — URL: <https://doc.qt.io/qt-5/> (дата обращения: 11.06.2026).
3. Справочная система C++ : [сайт]. — URL: <https://www.cplusplus.com/reference/> (дата обращения: 11.06.2026).

Приложение

Исходные коды всех модулей, включая заголовочные файлы, реализацию, файлы проектов Qt и примеры конфигурационных JSON-файлов, размещены в публичном репозитории на GitHub:

<https://github.com/tofficet>